

Static Type Checker Implementation

The static type checker is implemented as a pre-execution analysis pass over the Abstract Syntax Tree (AST). Instead of reusing runtime values, it introduces a dedicated `ASTType` interface (with methods like `isSubtypeOf` and `unfold`) and utilizes an `Environment<ASTType>` to track variable types in scope. Every AST node implements a `typecheck` method that evaluates its children and propagates the resulting type upward, throwing a `TypeError` if operations are invalid.

Key features of the type checker include:

- * **Scoping & Closures:** Block constructs like `let` and functions (`ASTFun`) create isolated environment scopes. Since closure parameters lack explicit type annotations, they are permissively bound to generic type variables (`TVar`) during the static pass.
- * **L-Value & R-Value Semantics:** To support stack-allocated mutable variables (`mut`), the system enforces implicit dereferencing when evaluating R-values, while strictly checking underlying reference types during L-value assignments.
- * **Memory Leak Prevention:** The checker statically prevents dangling pointers by analyzing block return types. If a local stack reference (`TRef` or `TMut`) attempts to escape the scope it was created in, the analyzer halts execution with a memory leak violation.

Recursive Types Extension

Recursive data structures are supported via an **equi-recursive** type system, meaning a recursive type and its single-step unfolded form are treated as entirely interchangeable.

This is implemented using two main components:

- * **TRec and TVar:** Recursive definitions are handled by a binding node (`TRec`) and a variable node (`TVar`).
- * **Lazy Unfolding:** When the type checker needs to compare types (e.g., during subtyping), `TRec` performs a lazy unfold. It calls a `substitute` method that searches its body and replaces any matching `TVar` with the parent `TRec` object itself.
- * **Structural Subtyping:** This unfolding mechanism pairs directly with `TEnum`, which enforces both Width subtyping (checking variant tags) and Depth subtyping (checking field arity and types). By implicitly unfolding `TRec` during these checks, the system safely verifies recursive custom struct tuples (like binary trees) on the fly without entering infinite loops.